

# Electron

- spawns backend ([main.py](#)) - includes safety checks including port check and single app instance check
  - Imports modules
  - Initializes FastAPI instance
  - create and store async queue for WebSocket messages
  - registers endpoints and logic (/health, /ws, /generate, startup)
  - runs Uvicorn engine (if `__name__ == "__main__"`)
    - binds backend to port :8000
  - runs startup logic:
    - Initialize ModelLoader class (imported `model_loader.py`) with queue
    - Grabs current event loop (`self.loop - asyncio.get_running_loop()`) before jumping into separate thread for blocking, memory-intensive image generation
- backend /health check `waitOn({ resources: ['http://127.0.0.1:8000/health']})`, then
- spawns frontend: Electron `createWindow` function loads `index.html` file packaged with `npm run build` in the Electron out folder
- input prompt, generate button sends request to /generate endpoint:
  - `await asyncio.to_thread(generate_image)` - send blocking image generation task to separate thread
    - grab stored ModelLoader instance from connections dict
    - run `model_loader.generate_image()`
    - `def progress_callback`
      - `self.loop.call_soon_threadsafe`: when safe for event loop, put progress update in queue for WebSocket

# A Timeline/Flowchart of App Startup and the Image Generation Process

## Understanding communication between Electron, Frontend (Next.js), Backend (Python)

Electron (main.js)

perform port check and single app instance check to ensure no existing app instances are running. Prevents communication mismatches between frontend and backend due to multiple launch instances. If clear, then:

Spawn backend - run [main.py](#) script:

- initialize FastAPI instance
- create async queue for WebSocket communication between frontend and backend (for progress bar updates and image cancellation request)
- register backend endpoints and logic (/health, /ws, /generate, startup)
- run Uvicorn engine - binds backend to port :8000
- run startup logic
  - grabs current event loop before jumping into separate thread for image generation, which blocks event loop and is memory-intensive
  - download StableDiffusion v1.5 from HuggingFace and assemble pipeline

check backend is running and model is ready - waitOn({ resources: ['http://127.0.0.1:8000/health']}). Then:

Spawn frontend - Electron creates desktop browser window running exported frontend (index.html)

Image Generation:

- user inputs prompt and clicks generate, sending request to /generate endpoint:
  - image generation task sent to separate thread to avoid blocking event loop and maintain WebSocket communication between frontend and backend, otherwise all progress updates would arrive after image generation is complete
  - at the end of each sampling step, a callback function puts progress update in queue for WebSocket in a thread-safe way
- image saved to disk and returned to user at frontend

App Cleanup: on exit or termination, backend server is killed and Electron app exits